Práctica personal: Cajero automático

Cuaderno de trabajo para pensar, probar y construir paso a paso.

Este espacio no busca quedar perfecto de una vez. Tú irás tomando decisiones, escribiendo tus ideas y probando pequeños avances; el acompañamiento te ayudará a revisar y a hacer nuevas preguntas.

Cómo usar este cuaderno

Completa solo una parte cada vez. Si algo no está claro, anótalo sin preocuparte por tener la respuesta inmediata. La idea es entender por qué funciona cada decisión.

Parte 1. Analicemos el problema

Imagina un cajero automático sencillo. La persona quiere retirar dinero, pero el programa debe revisar que el monto sea válido y que todavía haya saldo suficiente.

Dato de referencia	Valor del enunciado
Saldo inicial	₡250 000
Retiro mínimo	₡5 000
Retiro máximo	₡100 000

Preguntas guía para separar el problema

- ¿Qué debe ingresar la persona en cada turno?

Respuesta: Cuanto desea retirar

- ¿Qué debe validar el programa antes de aceptar un retiro?

Respuesta: Que la cantidad ingresada este en el rango de minimo ₡5 000 y maximo de ₡100 000

- ¿Qué cálculos necesita hacer el programa?

Respuesta: Restar el monto del retiro al saldo disponible cuando el retiro sea válido.

- ¿Cuáles datos son constantes y cuáles pueden cambiar?

Respuesta: Las constantes son el saldo inicial de ₡250 000, el retiro mínimo de ₡5 000 y el retiro máximo de ₡100 000. Las variables que cambian son el saldo disponible, el monto del retiro, la decisión de continuar y el contador de retiros exitosos

- ¿Qué cambia después de un retiro válido?

Respuesta: El saldo disponible disminuye según el monto retirado y el contador de retiros exitosos aumenta en uno.

- ¿Qué acciones se repiten y cuántas iteraciones imaginas?

Respuesta: Se repite la solicitud del monto, su validación, el mensaje del resultado y la pregunta de si desea continuar. El número de repeticiones depende de las decisiones de la persona y del saldo disponible

- ¿Qué variable podría controlar el ciclo?

Respuesta: La variable continuar controla si el ciclo sigue. El saldo disponible también puede utilizarse como condición para evitar repetir el proceso cuando ya no alcanza para el retiro mínimo

- ¿Cuándo termina el ciclo?

Respuesta: El ciclo termina cuando la persona decide no continuar o cuando el saldo disponible es menor que ₡5 000

- ¿Qué información se muestra al final o después de cada intento?

Respuesta: Después de cada intento, se muestra si el retiro fue aprobado o rechazado y el saldo disponible. Al final, se muestra el saldo restante y la cantidad de retiros exitosos

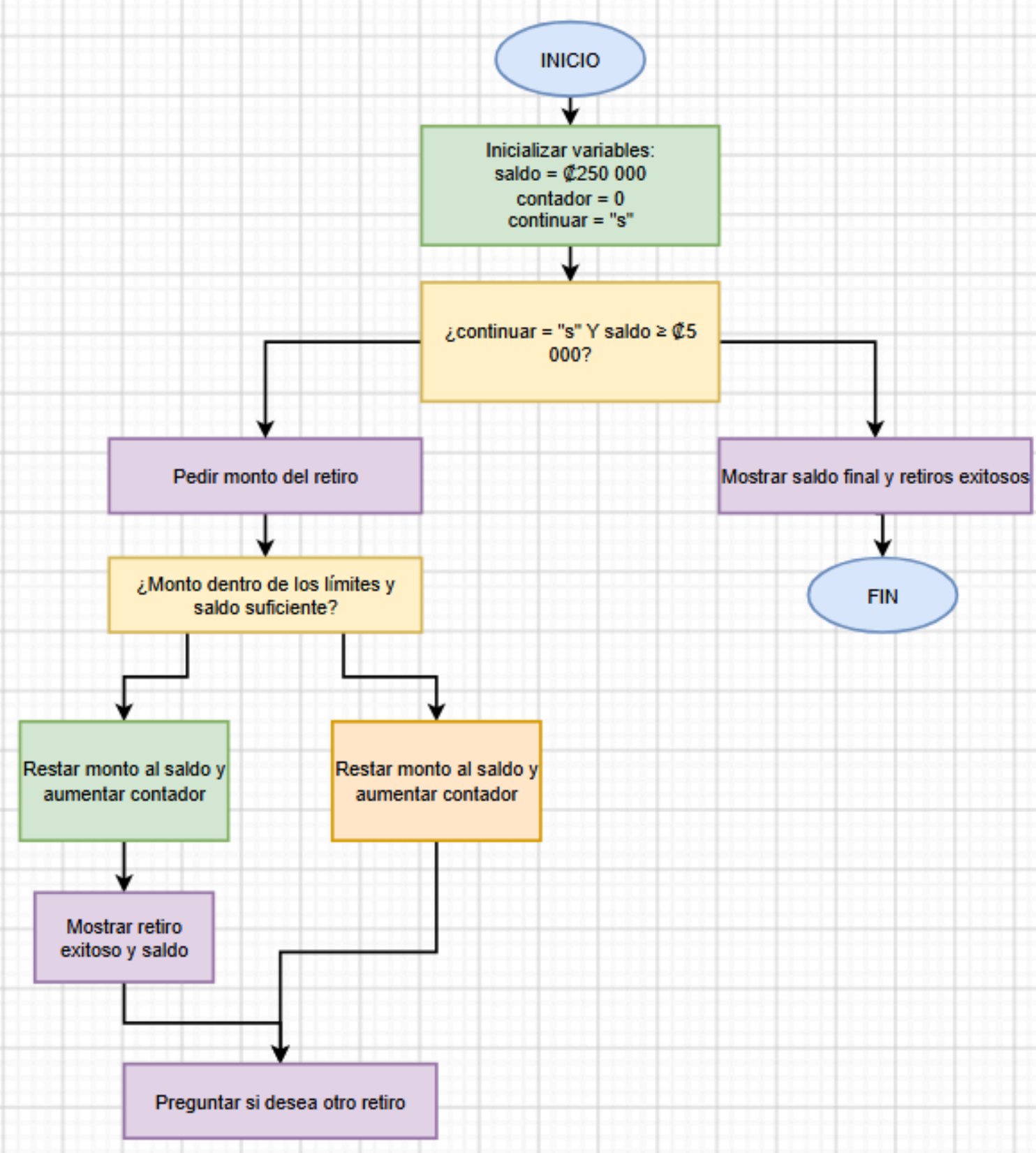
Mi primera idea

El cajero debe iniciar con un saldo de ₡250 000 y preguntar cuánto dinero desea retirar la persona. Después, debe comprobar que el monto esté dentro del mínimo y el máximo permitido y que haya suficiente saldo. Si el retiro es válido, debe restar el dinero y mostrar el saldo restante. Si no es válido, debe explicar el motivo. Luego, debe preguntar si la persona desea realizar otro retiro y repetir el proceso hasta que decida salir o ya no tenga saldo suficiente para retirar el mínimo.

Lo que todavía no tengo claro

Todavía no tengo claro cómo utilizar la variable que controla el ciclo while ni cómo hacer que el programa repita los retiros sin quedarse ejecutándose para siempre.

Parte 2. Diagrama de flujo



Parte 3. Tabla de casos de prueba

Antes de ejecutar el programa, calcula a mano qué debería ocurrir. Después escribe el resultado real y compara ambas columnas.

N.º	Tipo	Datos de entrada	Condición/iteración	Salida esperada calculada a mano	Resultado real	¿Coincide?
1	Retiro válido	Retirar ₡20 000; continuar: n	Monto válido y saldo suficiente	Retiro aprobado. Saldo: ₡230 000. Retiros exitosos: 1.	Retiro aprobado. Saldo: ₡230 000. Retiros exitosos: 1.	Sí.
2	Retiro menor que el mínimo	Retirar ₡3 000; continuar: n	El monto es menor que ₡5 000	Retiro rechazado. Saldo: ₡250 000. Retiros exitosos: 0.	Retiro rechazado. Saldo: ₡250 000. Retiros exitosos: 0.	Sí.
3	Retiro mayor que el máximo	Retirar ₡120 000; continuar: n	El monto supera ₡100 000	Retiro rechazado. Saldo: ₡250 000. Retiros exitosos: 0.	Retiro rechazado. Saldo final: ₡250 000. Retiros exitosos: 0.	Sí.
4	Saldo insuficiente	Retirar ₡80 000 cuando quedan ₡50 000	El monto está dentro de los límites, pero supera el saldo disponible	Retiro rechazado. Saldo: ₡50 000. Retiros exitosos: 0 en este intento.	Retiro rechazado. Saldo final: ₡50 000. Retiros exitosos: 2. El intento rechazado no aumentó el contador.	Sí.
5	Salida del ciclo	Retirar ₡10 000; continuar: n	La persona decide no continuar	Saldo final: ₡240 000. Retiros exitosos: 1.	Retiro aprobado. Saldo final: ₡240 000.	Sí.

					Retiros	

Nota para probar

Agrega otro caso si encuentras una situación importante. Por ejemplo, piensa qué ocurre cuando la persona decide no continuar o cuando intenta retirar exactamente el mínimo o el máximo.

Una observación antes de ejecutar

¿Qué caso crees que puede revelar un error en tu lógica?

El caso de saldo insuficiente, porque el monto puede estar dentro del mínimo y máximo, pero aun así ser mayor que el saldo disponible.

¿Qué cálculo hiciste a mano para comprobarlo?

Si el saldo disponible es ₡50 000 y la persona intenta retirar ₡80 000, el retiro debe rechazarse y el saldo debe permanecer en ₡50 000.

Parte 4. Python básico de consola

Aquí puedes escribir una primera versión pequeña, ejecutarla y mejorarla poco a poco. No necesitas resolver todo en el primer intento.

Requisitos de mi programa

- [ ] Declarar variables y constantes para el saldo inicial, el retiro mínimo y el retiro máximo.
- [ ] Pedir los datos por consola.
- [ ] Usar if/else para validar el retiro.
- [ ] Usar while para repetir el proceso.
- [ ] Usar un contador y decidir qué va a contar.
- [ ] Actualizar el saldo después de un retiro válido.
- [ ] Mostrar mensajes claros para la persona.

Secuencia incompleta para transformar en Python

1. **Preparar** Las constantes del saldo inicial, el mínimo y el máximo.
2. **Inicializar** El saldo disponible, el contador de retiros exitosos y la variable continuar.
3. **Mientras** La persona quiera continuar y el saldo alcance para el retiro mínimo.
4. **Pedir** El monto que la persona desea retirar.
5. **Si el retiro** Está dentro de los límites permitidos y hay saldo suficiente.

6. **En caso contrario** Mostrar un mensaje indicando que el retiro fue rechazado.
7. **Actualizar** El saldo restando el monto de un retiro válido.
8. **Preguntar** Si la persona desea realizar otro retiro.
9. **Aumentar** El contador cuando el retiro se completa correctamente.

CODIGO

Constantes

SALDO_INICIAL = 250000

RETIRO_MINIMO = 5000

RETIRO_MAXIMO = 100000

Variables iniciales

saldo = SALDO_INICIAL

contador = 0

continuar = "s"

Ciclo principal

while continuar == "s" and saldo >= RETIRO_MINIMO:

monto = int(input("¿Cuánto desea retirar? "))

Resultado final

print("Saldo final:", saldo)

print("Retiros exitosos:", contador)

Prueba 1

Entrada que usaré:

20000 y después n.

¿Qué espero que ocurra?

El retiro debe aprobarse, el saldo debe quedar en ₡230 000 y el contador debe indicar 1 retiro exitoso.

Prueba 2

Entrada que usaré:

3000 y después n.

¿Qué espero que ocurra?

El retiro debe rechazarse, el saldo debe permanecer en ₡250 000 y el contador debe permanecer en 0.

¿Qué error encontré?

En las primeras dos pruebas no encontré errores. El retiro válido descontó el dinero correctamente y el retiro menor que el mínimo fue rechazado sin modificar el saldo.

Parte 5. Revisión de estilo

Marca cada punto cuando lo revises. Si todavía no se cumple, conviértelo en un próximo paso y vuelve a probar.

- [x] ~~Nombres de variables claros.~~
- [x] ~~Constantes separadas de los datos que cambian.~~
- [x] ~~Sangría correcta.~~
- [x] ~~Mensajes comprensibles.~~
- [x] ~~Condiciones fáciles de leer.~~
- [x] ~~El while tiene una salida posible.~~
- [x] ~~El contador se actualiza en el lugar correcto.~~
- [x] ~~El saldo se actualiza solo cuando corresponde.~~
- [x] ~~Se validan el límite mínimo y el límite máximo.~~
- [ ] Se probaron casos normales y casos con error.
- [x] ~~Los comentarios son breves y solo aparecen donde ayudan.~~
- [x] ~~El código se mantiene sencillo para un nivel principiante.~~

Mi explicación breve

¿Qué parte del programa puedo explicar con más seguridad?

Puedo explicar cómo se valida el monto del retiro, cómo se actualiza el saldo y cómo se cuentan los retiros exitosos.

¿Qué parte todavía necesito revisar con acompañamiento?

Necesito practicar mejor cómo funciona el ciclo while y cómo se combinan las condiciones para decidir cuándo termina.

Una mejora que haré después

Me gustaría mejorar la validación para que el programa no se detenga si la persona escribe letras en lugar de un número.

Reto opcional. Segundo ciclo

Este reto es amable y completamente opcional. No necesitas hacerlo para terminar la práctica principal. Si quieres explorar un poco más, piensa en agregar un segundo ciclo para repetir todo el proceso o para permitir varios intentos de retiro inválido.

Preguntas para diseñarlo

- ¿Qué controla el primer ciclo?

Respuesta: Controla la repetición de los retiros mientras la persona quiera continuar y haya saldo suficiente.

- ¿Qué controlaría el segundo ciclo?

Respuesta: Controlaría los intentos para ingresar un monto válido cuando la persona se equivoca.

- ¿Qué contador usarías para cada ciclo?

Respuesta: El primer ciclo utiliza el contador de retiros exitosos. El segundo podría utilizar un contador de intentos inválidos.

- ¿Cuándo termina cada ciclo?

Respuesta: El primero termina cuando la persona decide salir o no hay saldo suficiente para el mínimo. El segundo termina cuando se ingresa un monto válido o se alcanza un límite de intentos.

- ¿Cómo evitarías confundir los dos ciclos?

Respuesta: Usaría nombres claros y asignaría una tarea diferente a cada ciclo.

Mi idea

¿Qué comportamiento nuevo quiero probar?

Quiero permitir que la persona corrija un monto inválido antes de pasar a la siguiente operación.

Mini diagrama

Iniciar cajero → Pedir monto → ¿Es válido? → Sí: actualizar saldo y contador → No: mostrar rechazo y volver a intentar.

Una prueba para el reto

Entrada o situación que probaré: ₡3 000 y después ₡20 000.

Resultado que espero observar: el primer monto se rechaza y el segundo se aprueba si hay saldo suficiente.

Lo que ocurrió realmente: pendiente de implementar y probar el reto opcional.

Nuestro acuerdo de trabajo 50/50

Este documento no busca quedar perfecto de una vez. Tú completas una parte, pruebas, anotas tus dudas y conservas la decisión sobre cómo quieres organizar tu código. El acompañamiento ayuda a revisar, hace preguntas y propone caminos para que puedas justificar tus propias decisiones.

La meta no es copiar una solución terminada, sino construir una explicación que puedas entender, modificar y volver a probar. Avanzamos juntos, pero tu voz y tu razonamiento siguen estando en el centro.

Para la próxima revisión

Trae una prueba, una duda o una decisión que quieras conversar. Incluso un error puede ser un buen punto de partida.

Mis preguntas finales

- ¿Qué entendí hoy?

Entendí cómo utilizar variables, constantes y condiciones para validar un retiro. También aprendí que el saldo solo cambia cuando el retiro es válido y que el contador registra los

retiros exitosos.

- ¿Qué necesito practicar?

Necesito practicar cómo funciona el ciclo while y cómo probar diferentes casos para encontrar errores.

- ¿Qué pregunta quiero traer a la próxima revisión?

¿Cómo puedo evitar que el programa se detenga cuando la persona escribe letras en lugar de un número?